

PRODUCT ARCHITECTURE OVERVIEW

BillPilot

AI copilot for telecom billing and operations

Ajay Kumar Kondeti

Software engineer · telecom billing · AI integration

Portfolio project · Mock BSS · **100% synthetic data**



Billing questions are costly, and failures surface too late

Common pain points across telecom billing, collections and operations (general industry patterns)

01

Bill shock & complaints

- Unexpected charges, partial-month proration and mid-cycle plan changes are hard to explain
- Customers contact care to understand a bill line by line

02

Slow disputes & credits

- Double charges, wrong rates, missed discounts, charges after cancellation
- Manual investigation across bills, usage and payments

03

Treatment & entitlement errors

- Accounts barred after paying, or treated during an open dispute
- Packs never activated, allowances that did not reset, charges for covered usage

04

Failures found too late

- Stuck bill runs, failed events and consumer lag spotted only after complaints
- Unbilled usage and fraud patterns caught late, if at all

BillPilot thesis: give every persona a grounded, auditable AI copilot over the billing stack - and keep a human in the loop for anything that moves money or service.

One platform: explain, resolve, operate

An AI assistant and operations layer on top of a TMF-shaped mock BSS



BUILT ON

TMF-shaped mock BSS APIs

Kafka event stream

PostgreSQL + synthetic data

RAG with citations

Human-in-the-loop approvals

Eval harness

Data policy: all customers, usage, bills and documents are synthetic. No real customer data and no proprietary vendor material.

Three role-based assistants, least-privilege by design

Each persona gets its own UI, tool allowlist and data scope



Customer

Chat assistant

NEEDS

- “Why is my bill higher this month?”
- “Was I charged twice? Can I get a credit?”
- “Roaming, refund, deposit and porting rules”

BILLPILOT GIVES

- Plain-language bill explanation
- Dispute intake with evidence
- Policy answers with citations
- Plan and VAS advice

Scope: Own account only • read + raise dispute



CSR

Troubleshooting console

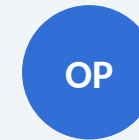
NEEDS

- “Paste an error, get a fix fast”
- “Investigate disputes and treatment”
- “Know what needs approval”

BILLPILOT GIVES

- Numbered steps cited from runbooks
- Account timeline across bill, usage, payments
- Proposed credits / unbar for approval

Scope: Assigned accounts • actions via approval queue



Ops team

Ops dashboard

NEEDS

- “Which events are failing right now?”
- “Are any bill runs stuck?”
- “Is the migration safe to commit?”

BILLPILOT GIVES

- Live Kafka failure dashboard
- Daily and monthly reports
- Approvals queue and migration control

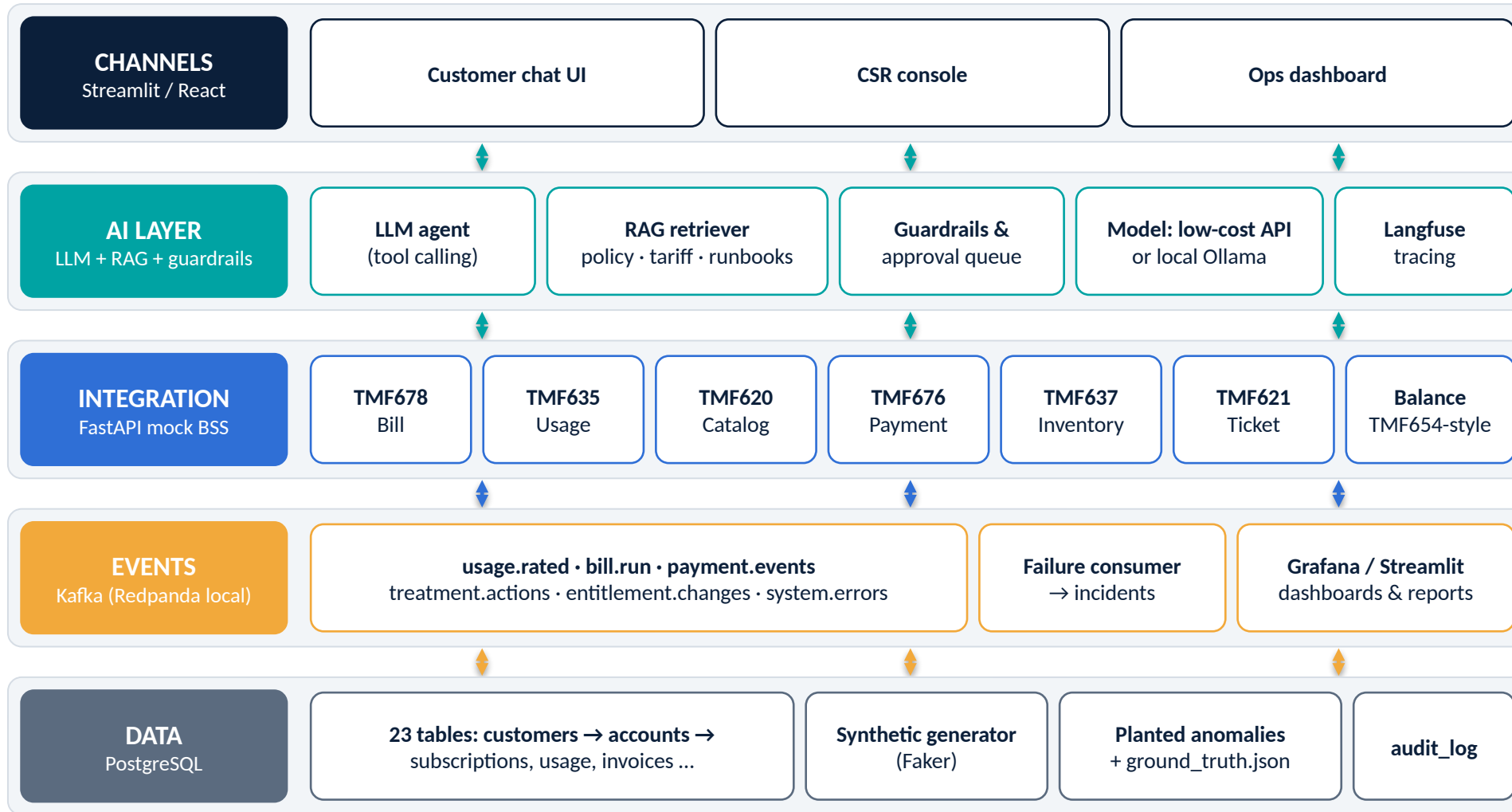
Scope: Aggregates + ops tools • no customer chat

13 capabilities across care, account integrity and operations



13 Built-in safety everywhere: human approval · audit logs · role-based access · rate limits

Layered, API-first, event-driven



CROSS-CUTTING

Security

RBAC per persona · human approval · audit log · rate limits

Observability

Langfuse traces · structured logs · incidents

Quality

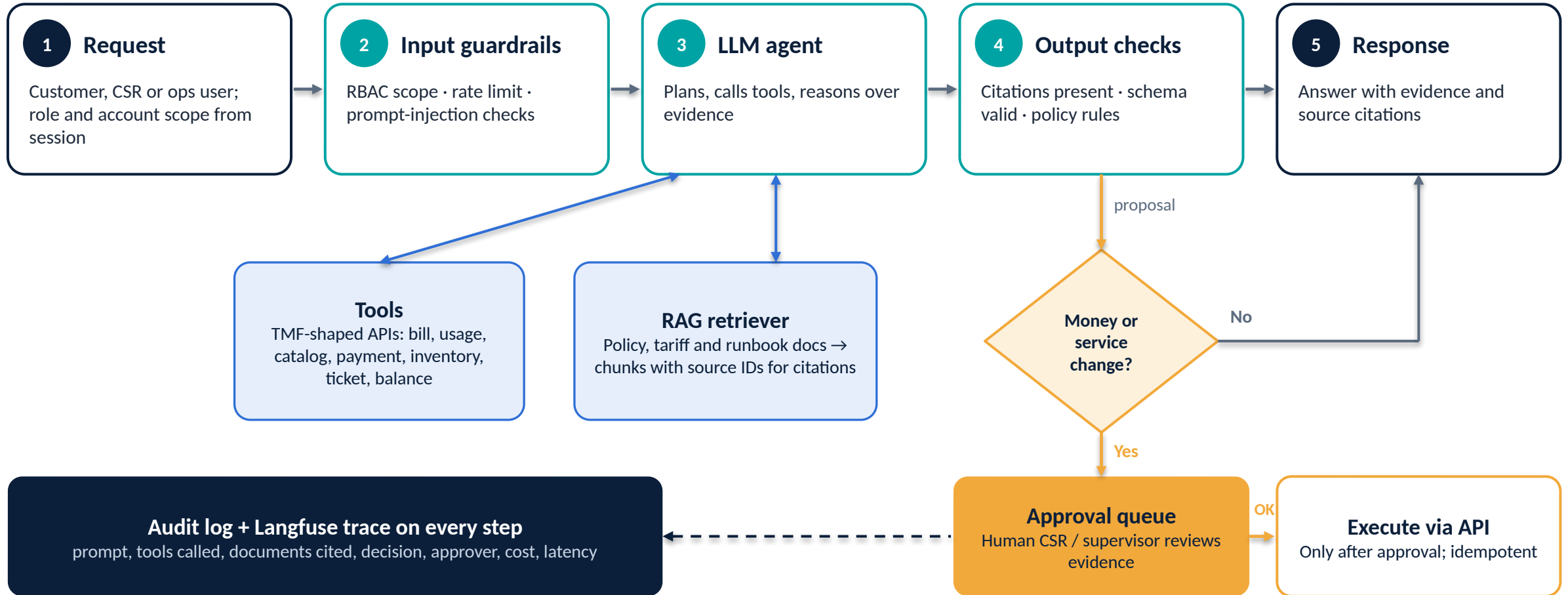
Eval harness: 50-100 labelled cases (target)

Delivery

Docker Compose · GitHub Actions CI · free-tier deploy

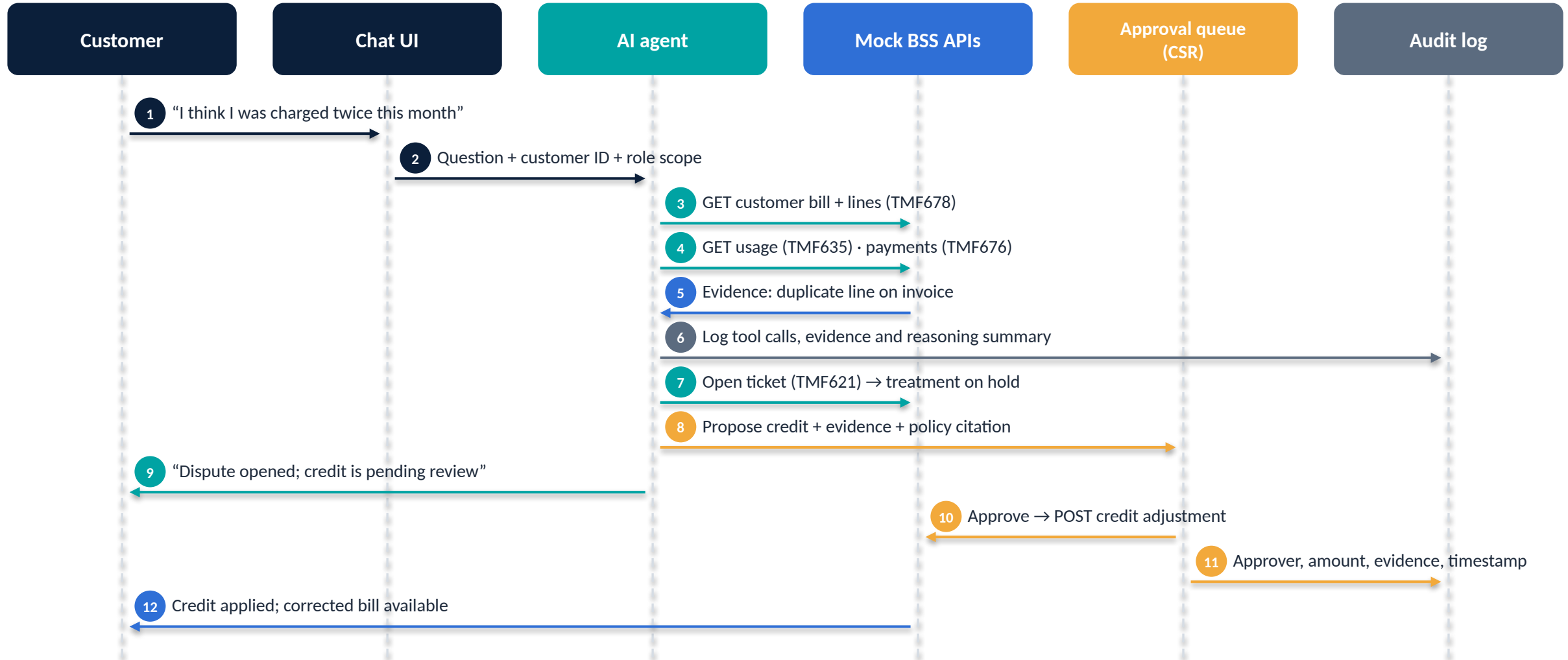
Request → tools + RAG → guardrails → approval → audit

The agent reads freely within its role; anything that changes money or service waits for a human



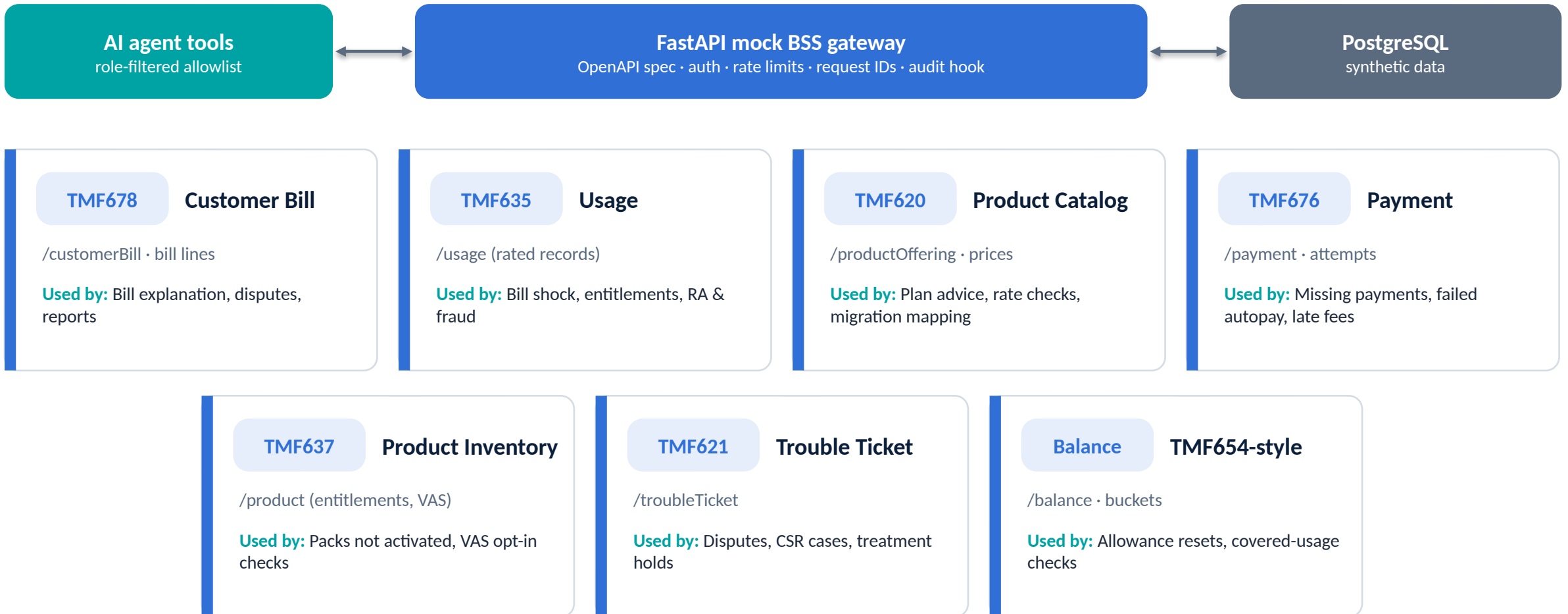
Dispute to credit: the AI proposes, a human approves

Example: a customer reports a double charge (synthetic scenario)



TMF-shaped mock BSS APIs

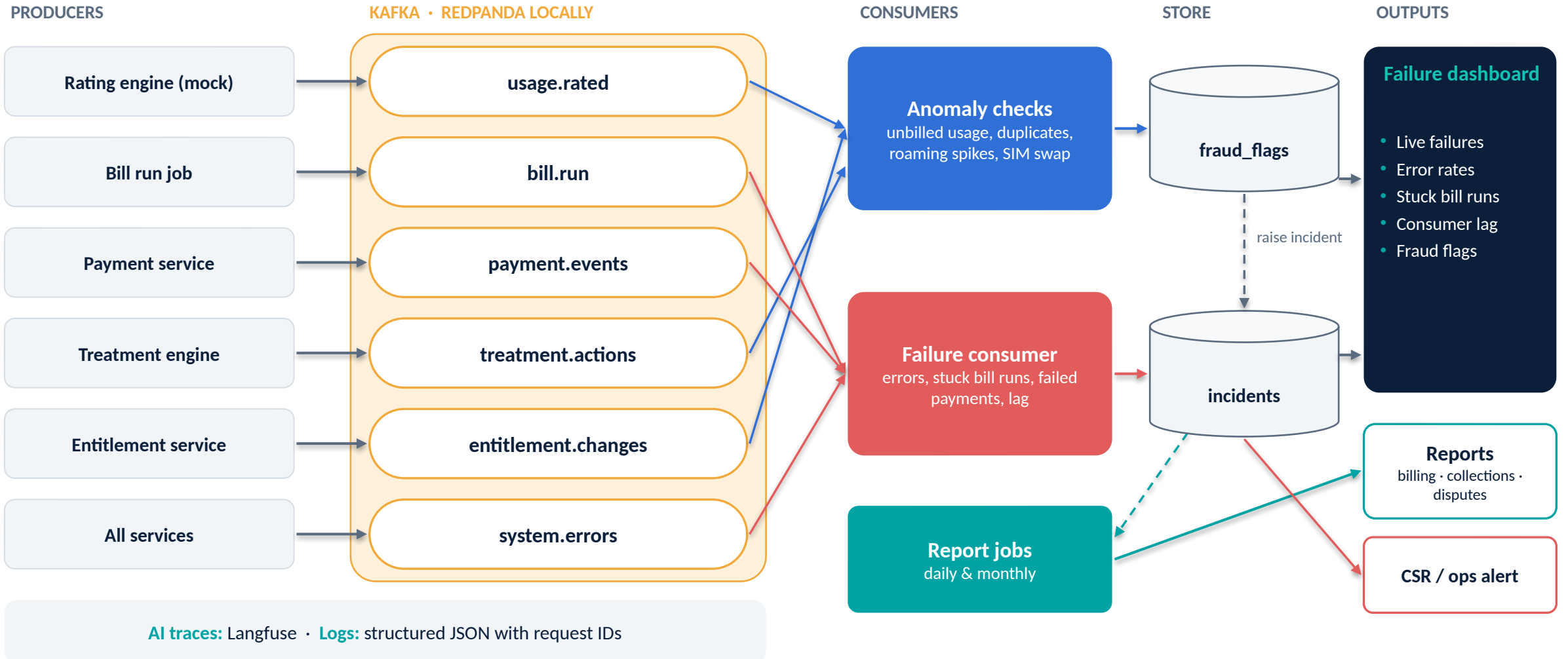
FastAPI services shaped like TM Forum Open APIs - the agent's only door into billing data



Resource names follow TM Forum Open API conventions; this is a learning mock, not a certified or conformant implementation.

Kafka topics to failure dashboard

Every billing step emits events; failures become incidents within the same pipeline



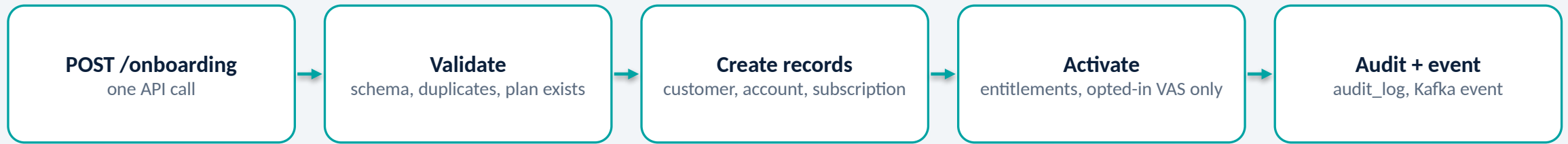
PostgreSQL data model: 23 tables in 7 domains

Account-centric schema; every money or service change also writes to audit_log

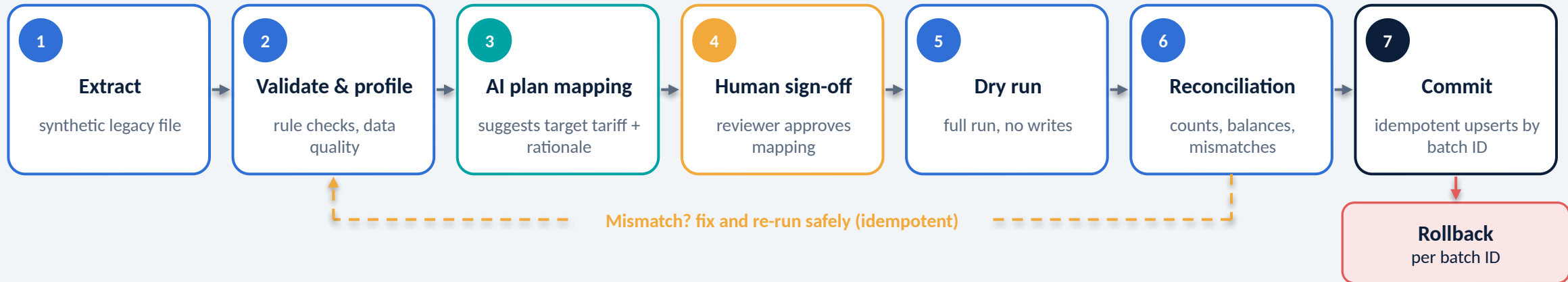


One API call for a new customer; a safe pipeline for bulk moves

SINGLE CUSTOMER ONBOARDING



BULK MIGRATION PIPELINE



Guarantees: nothing is written before dry run + sign-off • re-runs never duplicate records • every batch can be rolled back • AI suggests mappings, humans approve them

Safety is built into the architecture, not bolted on

1

Human approval

Credits, unbars, plan changes and disconnects wait in an approval queue

2

Audit log

Every prompt, tool call, citation, decision and approver recorded with request IDs

3

Role-based access

Per-persona tool allowlists and data scope; customers see only their own account

4

Rate limits & cost caps

Per-user and per-role request limits; token budget per conversation

5

Guardrails

Prompt-injection checks on input and retrieved text; output schema and citation checks

6

Data policy

Synthetic data only; secrets via environment variables; no proprietary material

ACTION RISK TIERS

READ

Auto

View bills, usage, payments, entitlements within role scope

ADVISE

Auto + citation

Explanations, policy answers, plan advice, troubleshooting steps

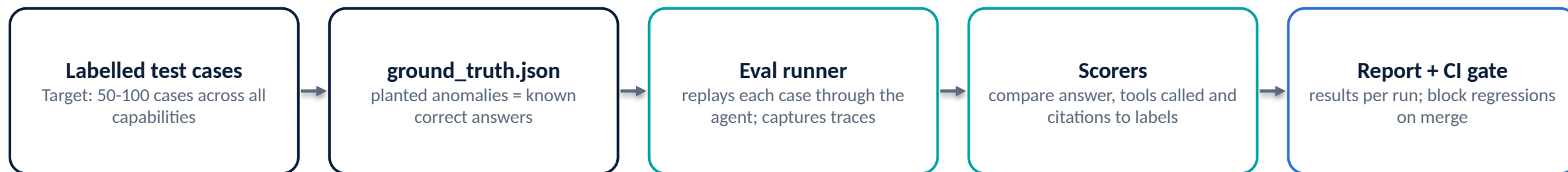
CHANGE

Human approval

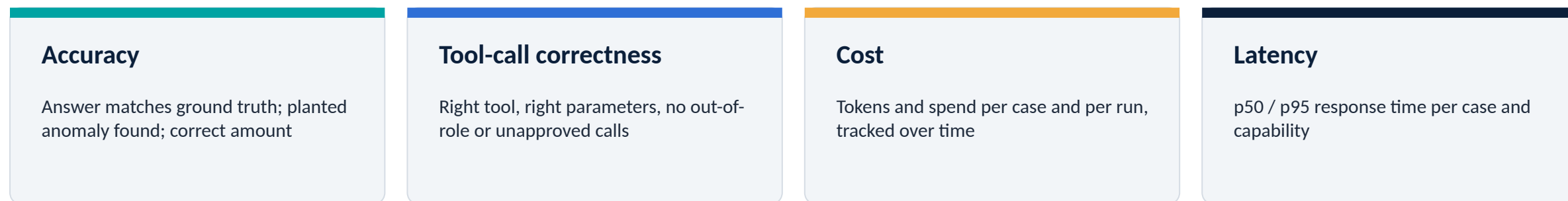
Credits, refunds, unbar / bar, plan or service changes, disconnects

Accuracy you can measure, because the answers are planted

An offline eval harness runs in CI against the synthetic ground truth



WHAT WE MEASURE



CASE MIX (PLANNED)

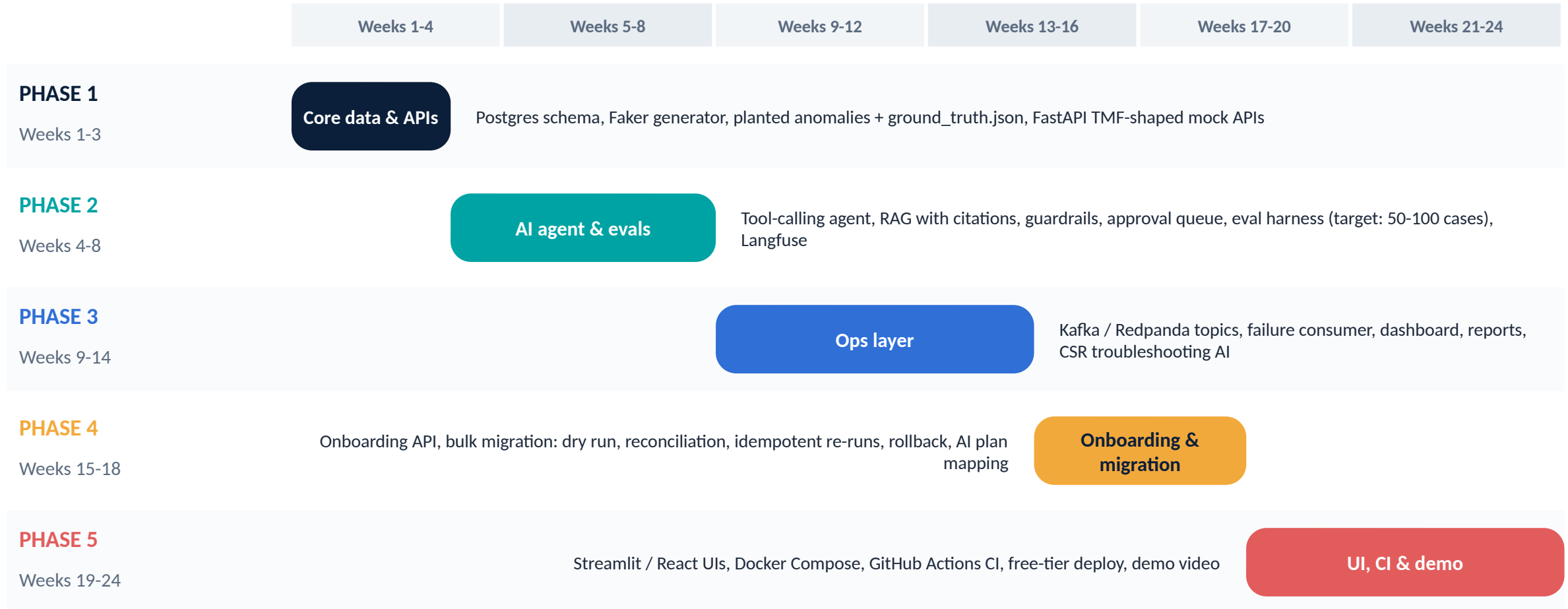


No results are claimed yet. Thresholds are set after a baseline run; every number reported will come from this harness.

What we chose, and why

DECISION	WHY
1 TMF-shaped APIs	Industry-standard vocabulary, credible to telco architects, no proprietary leakage
2 Planted anomalies + ground truth	Turns "seems to work" into measurable accuracy
3 Human-in-the-loop for money / service	Safety and auditability where errors cost customers
4 Event-driven failure monitoring	Operational visibility: find failures before customers do
5 Dry-run + idempotent migration	Safe onboarding: re-runs never duplicate, batches roll back
6 RAG with citations	Answers can be checked against source docs; fewer hallucinations
7 Role-based assistants	Least-privilege access: each persona sees only its own tools and data

Planned delivery over ~24 weeks (target plan)



Each phase ends with a working, demoable increment • *timeline is a target plan, not a commitment*

SUMMARY

BillPilot: grounded, auditable AI for telecom billing and operations

Telecom depth

Bills, disputes, treatment, entitlements, payments, RA & fraud - modelled on general industry patterns

AI integration

Tool-calling agent, RAG with citations, guardrails, approvals, Langfuse tracing

Engineering rigour

TMF-shaped APIs, Kafka events, idempotent migration, evals in CI, Docker deploy

Thank you

Ajay Kumar Kondeti · Repository and demo links to follow

Portfolio project · all data synthetic · no proprietary material